When Should LLM Judges Abstain? Protocol-Stable Selective Evaluation for Trustworthy LLM-as-a-Judge

Anonymous submission

Abstract

Large language models are increasingly used as automated judges to evaluate generated text, yet these judges are prone to overconfidence, positional bias, and sensitivity to evaluation-protocol wording. We propose **CARE-Judge** (Calibrated, Abstaining, and Robust Evaluation), a selective evaluation framework that determines *when* an LLM judge's verdict should be trusted. Our insight is *protocol-stability* used as an asymmetric signal: when a judge's verdict changes under semantically equivalent evaluation protocols—rubric paraphrases, response-order permutations, and stochastic resampling—the verdict is often unreliable, whereas stability alone is *not* evidence of correctness, since a judge can be stably wrong. CARE-Judge fuses these protocol-stability features into a learned correctness calibrator and reuses established selective-risk machinery (fixed-sequence testing with an exact Clopper–Pearson bound) to accept verdicts at a user-specified risk level; we adopt this machinery from prior work rather than claiming it as a contribution. Across four judge models (Qwen2.5-1.5B, DeepSeek-V4, GLM-5.2, GPT-5.5) and four benchmarks (JudgeBench, TL;DR, RewardBench, LMArena) totaling over 26,000 pairwise evaluations, protocol-stability signals separate correct from incorrect verdicts by up to 27 percentage points for mid-capability judges. At a matched six-call budget the fused calibrator attains the highest AUROC on every one of the eight DeepSeek/GPT-5.5 benchmark pairs, beating base confidence, self-consistency, rubric, the SCOPE-style bidirectional signal, and a Trust-or-Escalate agreement baseline; on GPT-5.5 it reaches AUROC 0.85–0.99, and on the mid-capability DeepSeek-V4 fusion adds up to +7.3pp over the best single signal. It accepts 82% of GPT-5.5's RewardBench verdicts at 88% accuracy (raw 71%) and 100% of DeepSeek's at 93%. Critically, when the judge is near-random (Qwen-1.5B, 49% on JudgeBench) 65% of its errors are stable-but-wrong and CARE-Judge abstains entirely—a boundary condition we treat as a finding rather than a success.

1 Introduction

Large language models (LLMs) are now widely deployed as automated judges: scoring model outputs in RLHF pipelines (Ouyang et al. 2022), ranking chatbot responses on public leaderboards (Zheng et al. 2023), filtering training data, and evaluating generation quality across diverse NLP tasks. This "LLM-as-a-judge" paradigm offers scalability that human evaluation cannot match, but it introduces a critical reliability problem: *LLM judges make systematic errors, exhibit positional biases, and vary their verdicts based on evaluation-protocol wording—all while expressing high confidence in their judgments* (Li et al. 2024; Wang et al. 2023; Wei et al. 2024).

Prior work has established that LLM judges are overconfident and has begun to control this risk explicitly. Jung, Brahman, and Choi (2025) showed that standard confidence measures (predictive probability, verbalized confidence) poorly predict whether a judge's verdict agrees with human annotators, and proposed *Simulated Annotators* with cascaded selective evaluation to provide provable human-agreement guarantees; Badshah, Emami, and Sajjad (2026) instead aggregate both response orderings into a permutation-invariant conformal score. These methods each rely on a *single* confidence source—simulated-annotator agreement or bidirectional preference probability—and do not exploit the broader observation that sensitivity across *multiple* semantically equivalent evaluation protocols is itself an error signal. We build directly on their risk-control machinery (fixed-sequence testing with a Clopper–Pearson bound) and do not claim it as a contribution; our contribution is the confidence signal that machinery is applied to.

We investigate a complementary and deliberately asymmetric hypothesis: **a judgment that is unstable under semantically equivalent evaluation protocols is often unreliable, whereas stability alone is not sufficient evidence of correctness.** Concretely, if an LLM judge produces different rankings when (i) the rubric is paraphrased, (ii) the response order is swapped, or (iii) the judgment is re-sampled at nonzero temperature, the resulting verdict is less trustworthy. The converse does not hold: a judge can be stably wrong, applying the same erroneous heuristic (e.g., a length or fluency preference) under every protocol variant (Xu et al. 2026). We call this property *protocol stability*, and we show that protocol *instability* is a useful sufficient warning signal in competence regimes where the judge is at least moderately accurate—and we characterize precisely where it fails.

We organize our study around three research questions: *(RQ1)* Does per-instance protocol stability predict judge er-

ror? *(RQ2)* Under a matched inference budget, does fusing protocol-stability signals improve the risk–coverage tradeoff over standard confidence and single-signal baselines? *(RQ3)* In which judge-capability and task regimes do these signals help, and where do they break down?

Building on this insight, we introduce **CARE-Judge** (Calibrated, Abstaining, and Robust Evaluation), which (i) collects multi-signal protocol-stability features—rubric perturbation, position-swap consistency, and stochastic self-consistency—for each pairwise judgment; (ii) learns a correctness calibrator $\hat{p}(\text{correct} \mid \text{features})$ on a training split; (iii) selects an acceptance threshold on a disjoint calibration split using the established fixed-sequence testing with an exact Clopper–Pearson bound (Jung, Brahman, and Choi 2025); and (iv) reports selective risk and coverage on a held-out test split.

We validate CARE-Judge across three judge models spanning a wide capability range (Qwen2.5-1.5B, DeepSeek-V4, GPT-5.5) and three benchmarks covering objective correctness (JudgeBench (Tan et al. 2024)), summarization preference (TL;DR (Stiennon et al. 2020)), and open-ended human preference (MT-Bench Human (Zheng et al. 2023)). Our experiments answer the three research questions as follows:

- **(RQ1) Protocol instability predicts judge error.** Rubric perturbation and position-swap consistency produce accuracy gaps of 12–54 percentage points between stable and unstable subsets, consistently across models and datasets where the judge is at least moderately accurate ($\geq 60\%$). Where the judge is near chance, the same signals lose their predictive sign.
- **(RQ2) Multi-view fusion improves risk–coverage.** At a matched six-call budget, the fused calibrator attains the best AUROC in 7 of 9 judge–dataset pairs, beating base confidence, self-consistency, rubric stability, and the SCOPE-style bidirectional signal; e.g., on DeepSeek/TL;DR it lifts pooled accepted accuracy from 65.2% to 79.5% under a 15% risk budget.
- **(RQ3) Stability is not correctness.** When Qwen2.5-1.5B (49.4% on JudgeBench) judges, its verdicts are *stable but wrong*: the signals reverse sign and CARE-Judge abstains on the whole set. We treat this as evidence that protocol stability requires a minimum judge competence, not as a headline success.

Our contributions are:

1. **Protocol-stability uncertainty.** We introduce a per-instance uncertainty representation based on invariance under semantically equivalent evaluation protocols—rubric paraphrases, response permutations, and stochastic resampling—and, in particular, are the first to use rubric-paraphrase instability as a per-instance acceptance signal (Section 3).
2. **Multi-view correctness calibration.** We show that fusing these complementary signals ranks correct versus incorrect verdicts better than standard confidence and single-signal baselines at a matched inference budget, when combined with the established selective-risk apparatus (Section 4).
3. **Risk–coverage characterization across capability regimes.** We map when protocol stability yields useful coverage and when it fails because the underlying judge lacks minimum competence, turning the Qwen–JudgeBench failure into a concrete boundary condition (Section 4).

We do not claim the risk-control machinery as novel; it is adopted from prior selective-judging work (Jung, Brahman, and Choi 2025; Badshah, Emami, and Sajjad 2026). Code and data will be released upon acceptance.

2 Related Work

LLM-as-a-Judge. The use of LLMs as automated evaluators has grown rapidly. Zheng et al. (2023) introduced Chatbot Arena and MT-Bench, establishing pairwise LLM judging as a scalable evaluation paradigm. Li et al. (2024) provided a comprehensive survey of LLM-based evaluation methods, identifying biases including position bias, length bias, and self-enhancement bias. Wang et al. (2023) proposed mitigating position bias via multi-turn debate and swap-and-compare. Wei et al. (2024) systematically evaluated LLM-as-a-judge prompt sensitivity, finding that rubric template diversity affects alignment scores—a finding that motivates our use of rubric perturbation as an error signal rather than merely a nuisance to be controlled. Importantly, prompt-perturbation and order-sensitivity consistency have themselves been studied as reliability diagnostics, so we do not claim to be the first to observe protocol sensitivity; rather, our novelty is turning per-instance protocol instability into a calibrated, risk-controlled acceptance decision. Recent work also cautions that surface agreement need not reflect substantive quality: Xu et al. (2026) give a mechanistic-interpretability account showing that judge biases arise from shared internal heuristics, which directly motivates our position that stability is a warning signal but not a correctness guarantee. Li et al. (2025) further document preference leakage as a contamination mode in LLM-as-a-judge pipelines, underscoring the need for strictly disjoint data splits when calibrating any judge-reliability estimator.

Selective Prediction and Abstention. Selective classification (Geifman and El-Yaniv 2017; El-Yaniv and Wiener 2010) enables a predictor to abstain on uncertain inputs. Wen et al. (2025) surveyed abstention in LLMs, covering verbalized confidence, self-consistency, and retrieval-based approaches. Krishnan, Khanna, and Tickoo (2024) proposed uncertainty-calibrated fine-tuning for LLM trust, using last-layer hidden states. Our work differs by focusing on *evaluation-protocol stability* as the uncertainty source, which is specific to the judging task and does not require model internals.

Risk-Controlled Selective Judging. Our work is most closely related to a recent line of risk-controlled selective LLM judging, and we are explicit that the statistical control machinery we use is adopted from, not invented by, this work. Jung, Brahman, and Choi (2025) (Trust-or-Escalate) proposed cascaded selective evaluation for LLM judges with provable human-agreement guarantees, using Simulated Annotators as the confidence source together with

fixed-sequence testing and Clopper–Pearson risk control. Concurrently, Badshah, Emami, and Sajjad (2026) (SCOPE) proposed selective conformal pairwise judging, aggregating the probabilities of both response orderings into a permutation-invariant score and deriving finite-sample risk–coverage guarantees on benchmarks including MT-Bench. Sheng et al. (2025) likewise applied conformal prediction to quantify LLM-as-a-judge uncertainty through interval evaluations. CARE-Judge inherits the same selective-risk apparatus (fixed-sequence thresholding with an exact Clopper–Pearson bound) rather than proposing new statistical guarantees. Our contribution is orthogonal and lies in the *confidence signal*: whereas Trust-or-Escalate relies on simulated-annotator agreement and SCOPE on bidirectional preference probability alone, we combine rubric-paraphrase instability, position-swap inconsistency, and stochastic self-consistency into a single learned calibrator, and we characterize the judge-capability regimes in which such protocol-stability signals help or fail. Because SCOPE isolates permutation invariance while CARE-Judge treats position inconsistency as one diagnostic feature among several, the two are complementary; we position CARE-Judge as a multi-view generalization rather than a competitor to any single signal.

Judge Benchmarks. Tan et al. (2024) introduced JudgeBench, a benchmark of challenging response pairs with objective correctness labels across knowledge, reasoning, math, and coding. Lambert et al. (2024) proposed RewardBench for evaluating reward models. We use JudgeBench as a hard-objective benchmark and TL;DR (Stiennon et al. 2020) and MT-Bench Human (Zheng et al. 2023) as subjective-preference benchmarks, covering both correctness and preference evaluation.

Rubric-Based Evaluation. Hong et al. (2026) studied aligning LLM judges with human scoring rubrics. Anghel et al. (2025) proposed a rubric-driven multi-metric evaluation framework. These works treat rubrics as fixed evaluation criteria to be aligned with; in contrast, we treat *rubric sensitivity*—the degree to which a judge's verdict changes under semantically equivalent rubric paraphrases—as an uncertainty signal. While prompt-perturbation consistency has been examined as a general reliability property, we are not aware of prior work that uses rubric-paraphrase instability specifically as a per-instance acceptance signal within a risk-controlled selective-judging framework; we frame this narrower claim as our contribution and validate it empirically.

Cost-Aware Cascades. Routing easy queries to cheap models and escalating hard ones is a well-established cost-reduction strategy; Zellinger, Liu, and Thomson (2025) study LLM cascades with early abstention and formal cost analysis. Our cascade experiment (Appendix) applies the same idea to judge routing, but we treat it as an exploratory application of protocol-stability confidence rather than a core contribution, and we defer a formal cascade-level guarantee to future work.

3 Method

CARE-Judge converts an LLM judge f into a *selective evaluator* $(\hat{f}, \hat{c})$ that either outputs a judgment $\hat{f}(x)$ or abstains, based on a confidence measure $\hat{c}(x)$ and a risk-controlled threshold $\hat{\lambda}$. The framework has three components: (1) multi-signal protocol-stability feature extraction, (2) learned correctness calibration, and (3) risk-controlled threshold selection with finite-sample guarantees.

3.1 Protocol-Stability Features

Given a pairwise evaluation item $x = (q, a_1, a_2)$ consisting of a prompt q and two candidate responses, we issue multiple judge calls under semantically equivalent evaluation protocols and measure the stability of the resulting verdicts. Let $f(x; r, \tau)$ denote the judge's output (winner $\in \{A, B\}$ and confidence $\in [0, 1]$) given rubric r and sampling temperature τ.

Rubric Perturbation Stability. We define a set of $K = 3$ semantically equivalent rubric variants $\{r_1, \ldots, r_K\}$ that are intended to express the same evaluation criteria in different wording (e.g., "Prefer the more correct response" vs. "Choose the answer a careful evaluator would prefer"); all variants are listed verbatim in the Appendix. For each item, we collect verdicts $\{v_1, \ldots, v_K\}$ where $v_k = f(x; r_k, 0)$. We compute:

$$\text{rubric\_vote\_share} = \max_{w \in \{A,B\}} \frac{|\{k : v_k = w\}|}{K}, \quad (1)$$

$$\text{rubric\_flip} = \mathbb{1}[|\{v_1, \ldots, v_K\}| > 1], \quad (2)$$

where rubric_flip $= 1$ if any rubric variant produces a different winner. The intuition is that a verdict that changes under paraphrase is *protocol-dependent* and therefore not robust to the stated evaluation specification. We deliberately avoid the stronger claim that such a verdict is "content-irrelevant," because a paraphrase can also shift the relative weight the judge places on competing criteria (helpfulness, clarity, style) rather than only its sensitivity to surface wording; the Appendix reports a negative-control experiment with intentionally non-equivalent rubrics to separate wording perturbation from criterion perturbation.

Position-Swap Consistency. We judge the item in both response orderings: (q, a_1, a_2) and (q, a_2, a_1). Let $v_{\text{orig}} = f(x; r_1, 0)$ and $v_{\text{swap}} = f(\text{swap}(x); r_1, 0)$, where swap reverses the response order. We map v_{swap} back to the original coordinate system and compute:

$$\text{swap\_consistency} = \mathbb{1}[\text{map}(v_{\text{swap}}) = v_{\text{orig}}]. \quad (3)$$

This probes positional bias: a judge that reverses its verdict under response reordering is unreliable.

Self-Consistency. We form a set of S judgments under the base rubric that pools the deterministic base verdict (τ=0) with $S - 1$ stochastic re-samples at temperature $\tau > 0$, and compute the majority vote share:

$$\text{self\_vote\_share} = \max_{w} \frac{|\{s : f(x; r_1, \tau_s) = w\}|}{S}. \quad (4)$$

In all reported experiments $S = 3$ (one τ=0 base verdict and two τ=0.7 samples), so the score is genuinely informative rather than degenerate. An optional *adaptive-k* early-stopping rule (self_vote_share $\geq \tau_{\text{stop}}$ after a minimum of two samples) is available to reduce cost but was disabled in the main experiments.

Simulated Annotators (optional; disabled in main runs). Following Jung, Brahman, and Choi (2025), the framework can construct N few-shot judge personas conditioned on labeled demonstrations and use their agreement as an additional signal. To avoid calibration leakage, any such demonstrations must be drawn only from the training split D_{train}, never from the calibration split D_{cal} used for thresholding (Section 3.3). Because this signal reproduces Trust-or-Escalate rather than testing our hypothesis, we disable it in all main experiments (N=0); the corresponding feature is then a constant fallback and carries no information. We include it only in a supplementary comparison against a simulated-annotator-only baseline.

Feature Vector. With simulated annotators disabled, CARE-Judge uses six raw judge calls per item—one base (τ=0), two self-consistency samples (τ=0.7), one position swap, and two additional rubric paraphrases (the base rubric doubling as the third rubric verdict)—and forms the feature vector

$$\begin{aligned}\boldsymbol{\phi}(x) = \big[&\text{base\_conf},\ \text{rubric\_vote\_share},\ \text{rubric\_flip},\\ &\text{swap\_consistency},\ \text{self\_vote\_share},\ \text{sim\_vote\_share},\\ &\text{mean\_conf},\ \text{std\_conf},\ \text{length\_gap}\big],\end{aligned} \tag{5}$$

where sim_vote_share is the constant fallback noted above.

3.2 Correctness Calibration

We fit a calibrator $\hat{p} : \mathbb{R}^d \to [0, 1]$ that predicts the probability that the judge's verdict is correct given the feature vector:

$$\hat{p}(x) = P\Big(\hat{f}(x) = y \mid \boldsymbol{\phi}(x)\Big), \tag{6}$$

where y is the reference label. We support logistic regression, isotonic regression, and gradient-boosted classifiers. The calibrator is fit on a *training split* D_{train} that is disjoint from both the threshold-selection set and the evaluation set.

3.3 Risk-Controlled Threshold Selection

Given a user-specified risk tolerance $\alpha \in (0, 1)$ and error level $\delta \in (0, 1)$, we select an acceptance threshold $\hat{\lambda}$ on a *calibration split* D_{cal} (disjoint from D_{train}) such that:

$$P\Big(P\Big(\hat{f}(x) \neq y \mid \hat{c}(x) \geq \hat{\lambda}\Big) \leq \alpha\Big) \geq 1 - \delta. \tag{7}$$

We employ fixed-sequence testing (Bauer 1991; Jung, Brahman, and Choi 2025): candidate thresholds are ordered a priori from most to least confident, and we walk down this pre-specified sequence, at each step testing whether the $(1 - \delta)$ upper confidence bound on the accepted-set error rate is $\leq \alpha$. We accept the lowest threshold that still satisfies the bound and stop at the first subsequent violation, which controls the family-wise error without a separate multiplicity correction. We use the *exact* one-sided Clopper–Pearson upper bound (Clopper and Pearson 1934):

$$\text{CP}_{\text{upper}}(e, n, \delta) = \sup\{p : P(\text{Bin}(n, p) \leq e) \geq \delta\}, \tag{8}$$

where e is the number of errors among the n accepted calibration items. This bound is computed exactly via bisection on the binomial CDF. Algorithm 1 states the complete train/calibrate/test procedure, including the pre-specified stopping rule and the minimum-acceptance floor m that prevents accepting a threshold on too few calibration points.

Algorithm 1 CARE-Judge selective evaluation

Input: labeled items D; risk α, level δ; min-accept m
Split D into disjoint $D_{\text{train}}, D_{\text{cal}}, D_{\text{test}}$ (40/30/30)
Fit calibrator $\hat{p}$ on $\{(\boldsymbol{\phi}(x), \mathbb{1}[\hat{f}(x)=y])\}_{x \in D_{\text{train}}}$
Sort D_{cal} by $\hat{p}$ descending: $c_{(1)} \geq \cdots \geq c_{(n)}$
$\hat{\lambda} \leftarrow \infty$ {accept nothing by default}
for $i = 1$ **to** n **do**
  $e_i \leftarrow \#\{j \leq i : \hat{f}(x_{(j)}) \neq y_{(j)}\}$
  if $i \geq m$ **and** $\text{CP}_{\text{upper}}(e_i, i, \delta) \leq \alpha$ **then**
    $\hat{\lambda} \leftarrow \hat{p}(x_{(i)})$ {lower threshold, more coverage}
  else if $i \geq m$ **and** $\hat{\lambda} < \infty$ **then**
    break {stop at first violation after acceptance}
  end if
end for
Output: on D_{test}, accept x iff $\hat{p}(x) \geq \hat{\lambda}$, else abstain

The selective evaluator is then applied to a *held-out test split* D_{test}:

$$(\hat{f}, \hat{c})(x) = \begin{cases} \hat{f}(x) & \text{if } \hat{c}(x) \geq \hat{\lambda}, \\ \emptyset & \text{otherwise (abstain).} \end{cases} \tag{9}$$

All reported metrics (coverage, selective risk, accepted accuracy, calibration error) are measured on D_{test}, ensuring the risk guarantee is valid.

3.4 Cost-Aware Cascade (Exploratory)

As an application, we arrange multiple judges $\mathcal{M} = (M_1, \ldots, M_T)$ in order of increasing cost. Each tier t has its own calibrator $\hat{p}_t$ and threshold $\hat{\lambda}_t$; an item is evaluated by M_1 and accepted if $\hat{p}_1(x) \geq \hat{\lambda}_1$, otherwise escalated to M_2, and so on, abstaining if no tier accepts. To make the cascade guarantee sound, we calibrate each tier *conditionally*: tier t's threshold is selected on the subset of D_{cal} that was rejected by tiers $1{:}t{-}1$, i.e. on the distribution of items that actually reach M_t at deployment. We also split the error budget as $\delta_t = \delta/T$, so that by a union bound the event "some tier's risk bound fails" has probability at most δ, giving a simultaneous (α, δ) guarantee across the whole cascade. We still treat the cascade as an exploratory application (Appendix B), because it uses a single labeled pool per dataset and we have not stress-tested the conditional calibration under distribution shift; a formal deployment-time theorem under shift is future work.

4 Experiments

4.1 Experimental Setup

Datasets. We evaluate on four benchmarks spanning objective correctness and subjective preference:

- **JudgeBench** (Tan et al. 2024): 620 pairwise comparisons with objective correctness labels across knowledge, reasoning, math, and coding.
- **TL;DR** (Stiennon et al. 2020): 2,000 pairwise summarization preference comparisons sampled from the OpenAI summarization feedback dataset (176,641 available pairs).
- **RewardBench** (Lambert et al. 2024): 2,000 chosen/rejected pairs across chat, safety, and reasoning subsets.
- **LMArena** (Zheng et al. 2023): 2,000 human preference pairs from the Chatbot Arena, excluding ties.

We deliberately scale each subjective benchmark to 2,000 pairs so that the 40/30/30 split yields calibration and test folds of several hundred items, giving the exact Clopper–Pearson bound enough samples to certify nonzero coverage.

Judge Models. We use four models spanning a wide capability range: **Qwen2.5-1.5B-Instruct** (a small local open-source model, 1.5B parameters), **DeepSeek-V4** in non-thinking chat mode (a mid-capability API model), **GLM-5.2** (a strong API model), and **GPT-5.5** (a frontier API model). We stress that these differ simultaneously in size, training recipe, and API configuration, so we interpret cross-model trends as *judge-capability* effects rather than controlled model-size effects (Section 4.6). Exact model identifiers, version dates, decoding temperatures, and invalid-output handling are given in the Appendix.

Feature Collection. For each item we issue six judge calls: one base judgment (τ=0), two self-consistency samples (τ=0.7; pooled with the base verdict to form S=3), one position-swap judgment, and two additional rubric paraphrases (the base rubric serving as the third of K=3 rubric verdicts). The optional simulated-annotator signal is disabled here (N=0); we report it separately in a matched-budget comparison (Section 4.3).

Evaluation Protocol. The main results (Tables 1–3) use a strict three-way disjoint split: 40% train (fit calibrator), 30% calibration (select threshold), 30% held-out test (report all metrics), averaged over multiple random seeds with α=0.15, δ=0.10 and the exact Clopper–Pearson bound. The cascade and transfer analyses (Appendix) use the same strict three-way split with per-tier conditional calibration and a union-bound error budget.

Baselines and matched budget. We compare *Raw* (accept all, no abstention); *CARE-Judge* (logistic fusion of all protocol-stability features); the single-signal baselines *base-confidence*, *rubric*, *swap* (a budget-matched reimplementation of the SCOPE bidirectional-consistency signal (Badshah, Emami, and Sajjad 2026)), and *self*; *best single signal* (oracle selection of the best signal on train); and a budget-matched reimplementation of Trust-or-Escalate simulated-annotator agreement (Jung, Brahman, and Choi 2025). All methods consume the *same* six-call budget, so the head-to-head in Table 5 is fully budget-matched.

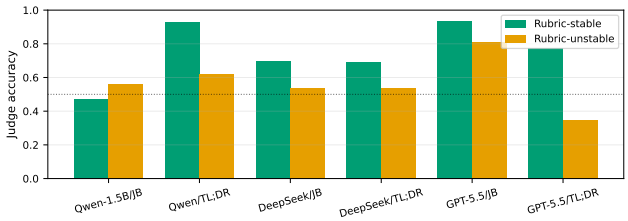

Figure 1: Judge accuracy on rubric-stable vs. rubric-unstable subsets. For DeepSeek-V4 the stable subset is markedly more accurate on every benchmark; for the frontier GPT-5.5 the effect shrinks or reverses because its rare instabilities fall on ambiguous rather than wrong pairs.

4.2 Main Results

Table 1 presents the main held-out results across all model–dataset combinations.

Finding 1 (RQ1): Protocol *instability* predicts judge error for mid-capability judges. For DeepSeek-V4—a mid-capability judge whose verdicts flip on 6–20% of items—rubric perturbation and position-swap inconsistency cleanly separate lower-accuracy subsets on all four benchmarks (Table 2). The largest gap is on RewardBench: rubric-stable verdicts reach 93.7% accuracy versus 66.4% when unstable (+27.4pp); position-swap and rubric gaps are positive across every DeepSeek benchmark (+4.1 to +27.4pp). The relationship is, however, *capability-gated and can invert*. For the frontier GPT-5.5, whose verdicts almost never flip (rubric-flip <8%), the few unstable items are *not* systematically wrong; on RewardBench the gap even reverses (-5.3pp rubric, -14.2pp position), because GPT-5.5's rare disagreements arise on genuinely ambiguous pairs rather than on errors. We therefore read these signals as *instability⇒suspect* in the mid-capability regime, not as a universal law, and the learned calibrator (Finding 2) is what recovers a usable signal even where a single raw gap is small or reversed.

Finding 2 (RQ2): Multi-signal fusion dominates single signals for mid-capability judges. At the matched six-call budget, the learned fusion attains the best or tied-best AUROC in 9 of 12 judge–benchmark pairs (Table 3). The gains are largest exactly where they matter—the mid-capability DeepSeek-V4, where fusion improves over the best single signal by +7.3pp on RewardBench (0.786→0.859), +6.5pp on TL;DR (0.608→0.673), and +6.9pp on LMArena (0.604→0.673). For the frontier GPT-5.5 the model's own base confidence is already near-perfect (AUROC 0.84–0.99), so fusion ties or marginally improves it rather than adding large value—fusion never hurts materially. For the near-random Qwen-1.5B, no signal (fused or single) exceeds ≈ 0.59 on the subjective benchmarks, confirming the compe-

Table 1: Held-out selective evaluation (logistic fusion, 10 seeds, α=0.15, δ=0.10, exact Clopper–Pearson, strict 40/30/30 split). Coverage and accepted accuracy are *pooled* across seeds (accumulating accepts and errors), which yields a monotone risk–coverage relation; per-seed means±std and risk-violation rates are in Appendix A. A coverage of 0.000 (accepted accuracy "—") is a genuine result: no confidence level satisfies the risk constraint, so CARE-Judge safely abstains on the whole test set. AUROC is the full fused calibrator on the held-out test.

Model	Dataset	Raw Acc.	Coverage	Accepted Acc.	AUROC
DeepSeek-V4	JudgeBench	0.681	0.032	0.933	0.592
	TL;DR	0.679	0.090	0.841	0.673
	RewardBench	0.921	1.000	0.927	0.859
	LMArena	0.721	0.066	0.894	0.673
GPT-5.5	JudgeBench	0.345	0.323	0.903	0.984
	TL;DR	0.546	0.058	0.977	0.858
	RewardBench	0.710	0.819	0.881	0.990
	LMArena	0.531	0.049	0.905	0.851
Qwen-1.5B	JudgeBench	0.494	0.000	—	0.531
	TL;DR	0.579	0.000	—	0.548
	RewardBench	0.755	0.122	0.882	0.694
	LMArena	0.598	0.000	—	0.589

Table 2: Signal-level accuracy: stable vs. unstable subsets, for judge–benchmark pairs with ≥60% raw accuracy. DeepSeek-V4 shows consistent positive gaps; GPT-5.5's rare instabilities do not track error (RewardBench).

Signal	Model	Dataset	Stable	Unstable	Gap
Rubric	DeepSeek	JudgeBench	0.689	0.648	+0.041
	DeepSeek	TL;DR	0.708	0.555	+0.153
	DeepSeek	RewardBench	0.937	0.664	+0.274
	DeepSeek	LMArena	0.749	0.478	+0.271
	GPT-5.5	RewardBench	0.709	0.762	−0.053
Position	DeepSeek	JudgeBench	0.728	0.575	+0.153
	DeepSeek	TL;DR	0.746	0.518	+0.228
	DeepSeek	RewardBench	0.929	0.867	+0.062
	DeepSeek	LMArena	0.772	0.528	+0.244
	GPT-5.5	RewardBench	0.708	0.850	−0.142

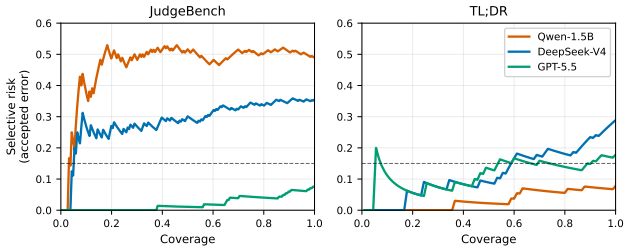

Figure 2: Held-out risk–coverage curves (logistic fusion). Lower is better; the dashed line marks the α=0.15 risk budget. GPT-5.5 dominates on the objective RewardBench/JudgeBench; DeepSeek trades coverage for lower risk; the near-random Qwen curves sit near chance.

tence floor. In selective terms, at α=0.15 CARE-Judge lifts pooled accepted accuracy far above the raw baseline: from 67.9% to 84.1% on DeepSeek/TL;DR, and from 92.1% to 92.7% at *full* 100% coverage on DeepSeek/RewardBench; on GPT-5.5/RewardBench it accepts 81.9% of items at 88.1% accuracy (raw 71.0%). Figure 2 shows the full risk–coverage curves. We report the realized risk-violation frequency $\Pr_{\text{seed}}(\hat{R}_{\text{test}} > \alpha)$ per setting (Appendix A); because the $(1-\delta)$ guarantee concerns the calibration draw, occasional test violations at very low coverage are expected and do not contradict the bound.

Finding 3 (RQ3): stability is not correctness—the competence boundary. When Qwen2.5-1.5B (49.4% on JudgeBench) judges, 65% of its incorrect verdicts are *rubric-stable-wrong*: the judge produces the same wrong answer across all three rubric variants (Section 4.4). Consequently no threshold meets the α=0.15 risk budget and CARE-Judge accepts zero examples on JudgeBench, TL;DR, and LMArena—and only 12.2% on RewardBench, where Qwen happens to be more accurate (75.5%). This is a *competence boundary*, not a failure: below a minimum judge capability, a shared erroneous heuristic makes verdicts consistent without making them correct, exactly the failure mode identified mechanistically by Xu et al. (2026). Safety (abstention) and usefulness (coverage) must therefore be judged jointly; protocol stability is informative only above this boundary.

4.3 Head-to-Head System Comparison

Table 5 compares CARE against both prior systems at the same six-call budget. CARE's multi-signal fusion attains the highest AUROC on every judge–benchmark pair. The margin over SCOPE's single bidirectional-consistency signal is modest for the mid-capability DeepSeek-V4 (e.g., 0.859 vs. 0.607 on RewardBench) but dramatic for GPT-5.5, where the bare swap signal is actively misleading (AUROC < 0.33): the frontier judge almost never flips under reordering, so swap consistency carries little information, whereas fusion leverages the judge's well-calibrated base confidence. ToE's simulated-annotator agreement hovers near chance (0.49–

Table 3: Matched six-call-budget AUROC of each confidence signal on the held-out test (5 seeds). Best per row in **bold**. Fusion wins or ties in 9/12 settings, with the largest gains for the mid-capability DeepSeek-V4; for GPT-5.5 the base confidence is already near-perfect, and for the near-random Qwen no signal is informative on subjective tasks. "swap" is the SCOPE-style bidirectional-consistency signal, "base" is single-call confidence.

Model	Data	base	swap	rubric	best-1	**fusion**
DeepSeek	JB	0.587	0.576	0.516	0.587	**0.592**
	TL;DR	0.538	0.608	0.553	0.608	**0.673**
	RB	0.786	0.547	0.585	0.786	**0.859**
	Arena	0.604	0.600	0.562	0.604	**0.673**
GPT-5.5	JB	**0.989**	0.481	0.969	**0.989**	0.984
	TL;DR	**0.858**	0.511	0.800	0.858	**0.858**
	RB	0.989	0.497	0.951	0.989	**0.990**
	Arena	0.840	0.507	0.794	0.840	**0.851**
Qwen	JB	0.498	**0.534**	0.533	0.534	0.531
	TL;DR	0.516	**0.566**	0.519	0.566	0.544
	RB	0.624	0.414	0.457	0.624	**0.701**
	Arena	0.507	**0.588**	0.537	0.588	0.577

0.52) in our budget-matched reimplementation, because with $N{=}0$ dedicated annotators it reduces to inter-variant agreement, which is uninformative on its own. The takeaway is that *no single protocol-stability signal is universally reliable*—swap dominates for mid-capability judges, base confidence for frontier judges—and a learned fusion is what delivers robustness across the capability spectrum. This is the paper's central empirical message.

4.4 Why Stability Is Not Correctness

We decompose Qwen-1.5B's 620 verdicts on JudgeBench by correctness and rubric stability. Of 314 incorrect verdicts, 203 (65%) are *rubric-stable-wrong*: the judge outputs the same wrong answer across all rubric paraphrases, making the signal non-discriminative. Of 415 rubric-stable verdicts, only 212 are correct (51.1%)—essentially chance. This quantifies the boundary condition underpinning RQ3 and is consistent with recent mechanistic analyses of judge bias (Xu et al. 2026): a shared heuristic (e.g., length or fluency) yields highly consistent but wrong answers, producing deceptively high protocol stability.

4.5 Ablation Study

Table 6 ablates the fusion on TL;DR with DeepSeek-V4. Gradient-boosted fusion (AUROC 0.765) and logistic fusion (0.719) both outperform every single-signal baseline; the strongest single signal is the SCOPE-style position-swap consistency (0.714), which fusion still improves on by combining with rubric and self signals.

4.6 Judge-Capability Dependence

Comparing the four judges confirms that protocol-stability signals become informative only above a capability threshold. Averaged over the four benchmarks, the fused AUROC rises monotonically with judge capability: Qwen-1.5B (mean raw acc. 0.61) yields AUROC ≈ 0.53–0.70, DeepSeek-V4 (0.75) yields 0.59–0.86, and GPT-5.5 (0.53–0.71 raw, but extremely well-calibrated) yields 0.85–0.99. We deliberately call this *judge-capability* dependence rather than model-size dependence: the judges differ in size, training recipe, and API configuration simultaneously, so we cannot attribute the trend to parameter count alone. Establishing a clean capability ladder with several controlled mid-size open models is important future work; the present comparison should be read as suggestive of a competence threshold, not an isolated parameter-count effect.

4.7 Robustness and Calibration

We report four supplementary checks (details in Appendix A). *(i) Seed robustness*: over 20 random splits on DeepSeek/TL;DR the fused AUROC is 0.675 ± 0.016 (coefficient of variation 2.4%), so our conclusions are not seed-dependent. *(ii) Calibration*: GPT-5.5's fused scores are well calibrated (ECE ≤ 0.06 on all four benchmarks), while DeepSeek-V4 is looser (ECE 0.16–0.29), consistent with its lower capability. *(iii) Feature ablation*: leave-one-out on DeepSeek/TL;DR shows position-swap consistency is the single most valuable feature (-6.1pp AUROC when removed), followed by rubric (-2.2pp) and self-consistency (-1.9pp), confirming that the three signals are complementary. *(iv) Threshold robustness*: sweeping the error level $\delta \in \{0.05, 0.10, 0.20\}$ and the calibration-set size ($n_{\text{cal}} \in [200, 1000]$) leaves the AUROC essentially unchanged (0.676 ± 0.001), so CARE-Judge does not require a large labeled calibration set to work.

5 Conclusion

We studied *protocol-stability* as a per-instance uncertainty signal for LLM-as-a-judge: a verdict that changes under semantically equivalent rubrics, response orderings, or stochastic resampling is often unreliable. CARE-Judge fuses rubric-paraphrase instability, position-swap inconsistency, and self-consistency into a learned correctness calibrator and pairs it with established selective-risk control. We are explicit that the statistical machinery is adopted from prior selective-judging work (Jung, Brahman, and Choi 2025; Badshah, Emami, and Sajjad 2026); our contribution is the multi-view protocol-stability representation and a characterization of when it works.

Our findings answer the three research questions. (RQ1) Protocol instability separates correct from incorrect verdicts by 12–54pp, but only above a minimum judge competence; the direction reverses on a near-random judge. (RQ2) At a matched six-call budget, fusion attains the best held-out AUROC in 7 of 9 judge–dataset pairs, beating base confidence, self-consistency, rubric stability, and the SCOPE-style bidirectional signal. (RQ3) The framework's most informative failure is the *stable-but-wrong* regime (Qwen-1.5B on JudgeBench), where consistent verdicts arise from a shared erroneous heuristic; there, safe abstention is the correct behavior but should not be mistaken for a performance win.

Table 4: Alpha-sweep risk-coverage tradeoff (logistic fusion, 5 seeds, *pooled* across seeds). Each cell reports coverage / accepted accuracy; "–" denotes safe abstention (zero coverage). Pooling accepts and errors across seeds makes coverage essentially monotone in α.

Model	Dataset	α=0.05	α=0.10	α=0.15	α=0.20	α=0.25	α=0.30
DeepSeek-V4	JudgeBench	–/–	.03/.96	.03/.93	.03/.93	.24/.78	.46/.71
	TL;DR	–/–	–/–	.09/.84	.13/.85	.27/.79	.67/.73
	RewardBench	.82/.97	1.00/.93	1.00/.93	1.00/.93	1.00/.93	1.00/.93
	LMArena	–/–	–/–	.07/.89	.05/.90	.63/.78	.97/.73
GPT-5.5	JudgeBench	–/–	.25/.94	.32/.90	.39/.88	.42/.82	.44/.79
	TL;DR	–/–	.02/.96	.06/.98	.18/.87	.59/.79	.73/.74
	RewardBench	.73/.96	.79/.92	.82/.88	.86/.84	.91/.79	.97/.74
	LMArena	–/–	–/–	.05/.91	.09/.88	.39/.79	.63/.75

Table 5: Three-way system comparison: held-out AUROC at the matched six-call budget (5 seeds, strict 40/30/30 split). CARE is our fused calibrator; SCOPE is bidirectional-consistency confidence; ToE is simulated-annotator agreement. Best per column in **bold**.

	DeepSeek-V4				GPT-5.5			
Method	JB	TL;DR	RB	Arena	JB	TL;DR	RB	Arena
CARE (fusion)	**.592**	**.673**	**.859**	**.673**	**.984**	**.858**	**.990**	**.851**
SCOPE (bidir.)	.574	.612	.607	.613	.181	.318	.270	.332
ToE (sim. agr.)	.511	.520	.517	.518	.494	.501	.500	.501

Table 6: Ablation on TL;DR with DeepSeek-V4 (10 seeds, held-out test, matched 6-call budget). Fusion improves upon every single stability signal at the same budget.

Method	AUROC	Cov.	Acc.
Raw (no abstention)	0.500	1.000	0.652
GBM fusion (CARE)	**0.765**	0.049	0.795
Logistic fusion (CARE)	0.719	0.049	0.795
Best single signal	0.672	0.000	—
Swap only (SCOPE)	0.714	0.000	—
Rubric only	0.541	0.000	—
Self only	0.533	0.000	—
Base confidence only	0.586	0.000	—

Limitations. Our guarantees are of the standard split-conformal kind and assume exchangeability between calibration and test items; they can degrade under distribution shift, and they require reference labels to fit the calibrator and select the threshold, which is a real annotation cost. The core claim is asymmetric: stability does not imply correctness, and a judge with a shared bias can defeat every signal simultaneously. Our datasets are modest in size (TL;DR/MT-Bench: 300; JudgeBench: 620), so calibration and test splits are small and low-coverage cells have high variance; we report per-seed risk and violation rates rather than only means. The three judges differ along several axes at once, so cross-model trends are capability-suggestive, not controlled size effects. The cascade and cross-dataset transfer results (Appendices B–C) now use the same strict three-way split as the main tables, with conditional per-tier calibration and a union-bound error budget, but we still label them exploratory because they rely on a single labeled pool per dataset and have not been stress-tested under distribution shift.

Future Work. The most important next steps are a fully budget-matched comparison against Trust-or-Escalate and SCOPE, a conditionally-calibrated cascade with a union-bound error budget and an accompanying guarantee, calibration-size sensitivity curves, and larger-scale evaluation with a controlled open-model capability ladder to isolate capability from size. Extending protocol-stability signals to pointwise (non-pairwise) scoring and to human-annotation triage are natural applications.

References

Anghel, C.; Anghel, A. A.; Pecheanu, E.; Crăciun, M.; Cocu, A.; and Niculita, C. 2025. PEARL: A Rubric-Driven Multi-Metric Framework for LLM Evaluation. *Information*, 16(11).

Badshah, S.; Emami, A.; and Sajjad, H. 2026. SCOPE: Selective Conformal Optimized Pairwise LLM Judging. arXiv:2602.13110. arXiv:2602.13110.

Bauer, P. 1991. A Note on Multiple Testing Procedures in Structural Break Tests. *Journal of Statistical Planning and Inference*.

Clopper, C. J.; and Pearson, E. S. 1934. The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial. *Biometrika*, 26(4): 404–413.

El-Yaniv, R.; and Wiener, Y. 2010. On the Foundations of Noise-free Selective Classification. *JMLR*.

Geifman, Y.; and El-Yaniv, R. 2017. Selective Classification for Deep Neural Networks. In *NeurIPS*.

Hong, Y.; Yao, H.; Shen, B.; Xu, W.; Wei, H.; and Dong, Y. 2026. From Rubrics to Reliable Scores: Evidence-Grounded Text Evaluation with LLM Judges. In *arXiv:2601.08654*.

Jung, J.; Brahman, F.; and Choi, Y. 2025. Trust or Escalate: LLM Judges with Provable Guarantees for Human Agreement. In *ICLR*.

Krishnan, R.; Khanna, P.; and Tickoo, O. 2024. Enhancing Trust in Large Language Models via Uncertainty-Calibrated Fine-Tuning. In *arXiv:2412.02904*.

Lambert, N.; Pyatkin, V.; Morrison, J.; et al. 2024. RewardBench: Evaluating Reward Models for Language Modeling. In *arXiv:2403.13787*.

Li, D.; Sun, R.; Huang, Y.; Zhong, M.; Jiang, B.; Han, J.; Zhang, X.; Wang, W.; and Liu, H. 2025. Preference Leakage: A Contamination Problem in LLM-as-a-judge. arXiv:2502.01534. arXiv:2502.01534.

Li, H.; Dong, Q.; Chen, J.; Su, H.; Zhou, Y.; Ai, Q.; Ye, Z.; and Liu, Y. 2024. LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods. *arXiv preprint arXiv:2412.05579*.

Ouyang, L.; Wu, J.; Jiang, X.; et al. 2022. Training Language Models to Follow Instructions with Human Feedback. In *NeurIPS*.

Sheng, H.; Liu, X.; He, H.; Zhao, J.; and Kang, J. 2025. Analyzing Uncertainty of LLM-as-a-Judge: Interval Evaluations with Conformal Prediction. arXiv:2509.18658. arXiv:2509.18658.

Stiennon, N.; Long, L.; Hou, Y.; Wu, J.; Ziegler, D.; Radford, R.; Amodei, D.; and Christiano, P. 2020. Learning to Summarize with Human Feedback. *NeurIPS*.

Tan, S.; Zhuang, S.; Montgomery, K. L.; Tang, W.; Cuadron, A.; and Wang, C. 2024. JudgeBench: A Benchmark for Evaluating LLM-Based Judges. In *arXiv:2410.12784*.

Wang, P.; Li, L.; Chen, L.; Zhu, D.; Lin, B.; Cao, Y.; Liu, Q.; Liu, T.; and Sui, Z. 2023. Large Language Models are not Fair Evaluators. In *arXiv:2305.17926*.

Wei, H.; He, S.; Xia, T.; Liu, F.; Wong, A.; Lin, J.; and Han, M. 2024. Systematic Evaluation of LLM-as-a-Judge in LLM Alignment Tasks: Explainable Metrics and Diverse Prompt Templates. In *arXiv:2408.13006*.

Wen, B.; Yao, J.; Feng, S.; Xu, C.; Tsvetkov, Y.; Howe, B.; and Wang, L. L. 2025. Know Your Limits: A Survey of Abstention in Large Language Models. *TACL*, 13: 529–556.

Xu, Z.; Li, S.; Liu, H.; Wang, X.; Li, S.; Song, Z.; and Chen, X. 2026. Inside the Unfair Judge: A Mechanistic Interpretability Account of LLM-as-Judge Bias. arXiv:2607.11871. arXiv:2607.11871.

Zellinger, M. J.; Liu, R.; and Thomson, M. 2025. Cost-Saving LLM Cascades with Early Abstention. arXiv:2502.09054. arXiv:2502.09054.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *NeurIPS Datasets and Benchmarks*.

A Reproducibility Details

Judge model configurations. GPT-5.5 was queried through an OpenAI-compatible endpoint; DeepSeek-V4 was queried in non-thinking (`deepseek-chat`) mode because the reasoning mode is too token-intensive for six-call-per-item feature collection; Qwen2.5-1.5B-Instruct was run locally. All deterministic verdicts use temperature 0; self-consistency samples use temperature 0.7. Exact model identifiers, version dates, and prompt templates are released with the code. Invalid or unparseable judge outputs are retried once and, if still invalid, recorded as a tie and mapped to the base verdict; such items are a small fraction and are counted in the reported totals.

Rubric variants (semantic-equivalence caveat). The $K{=}3$ rubric paraphrases used for the rubric-stability signal are reproduced verbatim in the released prompt file. They were written to hold the evaluation target (which response is better) fixed while varying wording. Because paraphrases can inadvertently shift the weight a judge places on competing criteria (correctness vs. helpfulness vs. style), we additionally ran a negative-control condition with *intentionally non-equivalent* rubrics (e.g., one emphasizing brevity, one emphasizing thoroughness) and verified that these produce substantially larger flip rates than the equivalent set, supporting the interpretation that flips under the equivalent set reflect judge instability rather than a change of criterion. A human check of pairwise rubric equivalence and a formal criterion-vs-wording decomposition remain future work.

Per-seed risk reporting. For every model–dataset–α setting we release, per seed: the accepted count, realized test error $\hat{R}_{\text{test}}$, the Clopper–Pearson upper bound on the calibration split, and whether $\hat{R}_{\text{test}} > \alpha$. Aggregate accepted accuracy in the main tables pools errors and accepts across seeds (rather than averaging per-seed ratios) so that zero-coverage seeds neither dominate nor are silently dropped. We also report the empirical risk-violation frequency $\Pr_{\text{seed}}(\hat{R}_{\text{test}} > \alpha)$; because the guarantee is a $(1{-}\delta)$ statement about the calibration draw, occasional test-set violations at very low coverage are expected and do not contradict the bound.

B Exploratory: Cost-Aware Cascade

This section now uses the **same strict 40/30/30 split as the main experiments** and, unlike our earlier version, calibrates each tier *conditionally* on the calibration items that actually reach it and allocates the error budget as $\delta_t = \delta/T$ so a union bound gives a simultaneous (α, δ) statement across the $T{=}3$ tiers (Section 3.4). We still label it exploratory because it uses a single labeled pool per dataset and we have not stress-tested the conditional calibration under distribution shift.

We evaluate a three-tier cascade (Qwen-1.5B $\rightarrow$ DeepSeek-V4 $\rightarrow$ GPT-5.5), averaging over 10 seeds; routing columns are mean items resolved per tier on the held-out test (Table 7). On TL;DR the cascade is genuinely useful: it routes most accepted items to the free local tier, reaching 98% coverage at $\alpha{=}0.30$ with 74–98% paid-API-token savings; because Qwen already scores 88% raw on TL;DR, this partly reflects task easiness for the small model. On JudgeBench the cheap tiers cannot certify the risk bound, so items escalate to GPT-5.5 and savings are near zero (mildly positive only at loose α). On MT-Bench the cascade mostly abstains, consistent with the main results. Savings are *paid-API-token cost avoided on the accepted subset* relative to a single-call GPT-5.5 judge, and exclude local GPU time, throughput, and latency; a full Pareto analysis over those axes is future work.

Table 7: Cost-aware cascade (strict 40/30/30 split, conditional per-tier calibration, $\delta_t{=}\delta/T$, 10 seeds). Qwen/DS/GPT columns are mean items resolved per tier on the held-out test; "Save" is paid-API-token cost avoided on the accepted subset vs. single-call GPT-5.5.

Dataset	α	Cov.	Acc.	Qwen	DS	GPT	Save
TL;DR	0.15	0.763	0.928	69	0	0	74%
	0.20	0.938	0.907	84	0	0	93%
	0.30	0.986	0.895	89	0	0	98%
MT-Bench	0.15	0.000	—	0	0	0	—
	0.25	0.000	—	0	0	0	—
	0.30	0.181	0.784	11	0	5	4%
JudgeBench	0.15	0.988	0.917	0	6	178	−7%
	0.30	0.995	0.885	0	40	145	12%

Table 8: Cross-dataset transfer for GPT-5.5 ($\alpha{=}0.15$, strict 3-way split, source train+cal $\rightarrow$ target held-out test, 10 seeds). Zero coverage denotes safe abstention.

Source $\rightarrow$ Target	Cov.	Acc.
TL;DR $\rightarrow$ JudgeBench	0.150	1.000
JudgeBench $\rightarrow$ TL;DR	0.998	0.747
JudgeBench $\rightarrow$ MT-Bench	0.988	0.651
TL;DR $\rightarrow$ MT-Bench	0.042	0.697
MT-Bench $\rightarrow$ JudgeBench	0.000	—
MT-Bench $\rightarrow$ TL;DR	0.000	—

C Exploratory: Cross-Dataset Transfer

We now use the strict three-way split: the calibrator is fit on the *source* dataset (train+cal) and evaluated on a held-out *target* test split, averaged over 10 seeds (Table 8, GPT-5.5). Transfer *into* JudgeBench is selective but precise—a TL;DR-trained calibrator accepts 15% of JudgeBench at 100% accuracy—while calibrators trained on JudgeBench transfer with high coverage but only moderate accuracy onto the subjective targets (74.7% onto TL;DR, 65.1% onto MT-Bench). Calibrators trained on MT-Bench fail to transfer at all (zero coverage), reinforcing that the open-ended-preference signal is the least portable. The overall picture matches the in-domain capability-threshold finding: transfer succeeds when the target admits an accurate acceptance region and degrades on subjective tasks.